SPM "One-Shot" Project

SPM course a.y. 25/26

Iterative Sparse Matrix-Vector Computation on Evolving Sparse Matrices

Project Overview

In this project you will study an iterative sparse matrix-vector computation in shared-memory and distributed-memory settings. Given a sparse matrix $A \in \mathbb{R}^{N \times N}$ and an initial vector x_0, the computation repeatedly applies a sparse matrix-vector product followed by a normalization step. Every fixed number of iterations (epoch), the matrix rows are circularly shifted. The matrix remains sparse, its size remains fixed, and the total number of non-zero elements remains constant.

The goal is not simply to parallelize a sparse matrix-vector product, but to understand how sparsity, workload imbalance, matrix evolution, task granularity, communication, and data redistribution affect performance. The quality of the analysis is a central part of the evaluation.

Sequential Reference Implementation

A sequential reference implementation is provided (iterative_SpMV.cpp). You must use this implementation as the reference for the algorithm and for correctness purposes. If you improve the provided sequential implementation, you must use that same improved sequential version as the baseline for performance comparisons. The purpose is to compare execution models, not different algorithms.

Required Implementations

You must provide: 1) C++ threads implementation; 2) OpenMP implementation based on tasks; 3) MPI+OpenMP implementation.

The C++ and OpenMP versions must be executed on a single node. The MPI+OpenMP version must be executed on multiple nodes of the spmcluster. You may optionally provide an additional OpenMP implementation based on work-sharing constructs and compare it with the task-based version.

C++ Threads and OpenMP Implementations

The C++ threads and OpenMP implementations must run on a single node and preserve the same algorithmic structure as the sequential reference implementation. The report must discuss the adopted work distribution strategies and how they behave on the irregular evolving workload. It must also analyze the granularity of the units of work used in both the C++ threads implementation and the OpenMP task-based implementation.

If you also implement a work-sharing OpenMP version, use it only as an additional comparison point.

MPI+OpenMP Implementation

The distributed version must combine MPI across processes and OpenMP within each process. In the distributed version, the matrix is initially generated by the master process and distributed across MPI processes. At epoch changes, the distributed implementation must correctly reproduce the matrix evolution. Depending on the chosen data layout, this may involve redistributing matrix rows or updating the distributed row mapping. The design of the distributed decomposition is part of the project. Different choices may lead to different performance results. You are expected to justify your choices through measurements and analysis.

Correctness

All implementations must reproduce the numerical result of the sequential reference implementation. Due to floating-point arithmetic, parallel implementations are not required to produce bitwise-identical results. For small inputs, stronger checks may be performed. For larger inputs, correctness must be verified by comparing the final Rayleigh-like value and, when requested, the final vector against the sequential reference within a reasonable numerical tolerance. The adopted correctness criterion must be described in the report. The

provided implementations must support an optional vector dump mode for correctness verification. Vector dumping and verification must be kept separate from the measured region.

Performance Evaluation

You must evaluate the performance of your implementations on the spmcluster nodes. The experimental evaluation should focus on meaningful configurations. Extensive parameter sweeps are not required, but the selected experiments must be sufficient to support a convincing analysis.

For the C++ threads and OpenMP implementations, evaluate performance on a single node using the irregular workload. The report must compare the adopted work distribution strategies and include an analysis of task granularity. For the MPI+OpenMP implementation, the report must include:

- strong and weak scalability results
- speedup with respect to the sequential baseline
- analysis of the interaction between MPI processes and OpenMP threads
- a detailed breakdown of execution time, including at least:
    - local computation time
    - communication time during the iterations
    - global reduction time
    - epoch transition time, including matrix redistribution or logical data-layout updates
    - total execution time

You are expected to discuss the trade-off between local computation, communication, reductions, and redistribution costs. All experiments must clearly state: number of nodes and MPI processes used, number of OpenMP threads.

Notes: You are expected to make reasonable engineering choices and justify them. There is no single correct parallelization or distribution strategy, provided that correctness is preserved, the overall algorithmic structure remains recognizable, and the adopted choices are justified through analysis and experiments. A central goal of this project is to understand how distributed performance depends on local computation, communication, and data (re-)distribution.

Deliverables and deadlines

Send the teacher an email with subject "SPM Project" and attach a zip file named *SPMProject_NameSurname.zip* containing the source code and the report. The project must be sent by the deadline of the exam session in which you intend to take the exam. The deadline reported on the "esami" portal (https://esami.unipi.it/) must be interpreted as the project submission deadline for that exam session, not as the exam date. After the submission deadline you will receive an email with the available exam dates. The project remains valid for the entire academic year. Exam registration is mandatory.

The zip file must contain:

- source code and build instructions, either a Makefile or explicit compile commands
- a README containing instructions on how to run the tests and perform correctness checks
- a PDF report, max 15 pages, that includes:
    - implementation strategies adopted
    - OpenMP task-based strategy and granularity analysis
    - MPI+OpenMP distribution strategy
    - correctness verification
    - strong and weak scalability results
    - performance breakdowns
    - discussion of the results obtained
    - analysis of the main scalability bottlenecks

  The report should focus on analysis rather than code description. The code should be properly structured and commented.